Wearable Baselines on a Public Ledger: Portable, Owner-Controlled Calibration for Cross-App BCI and Biosignal Interfaces

Alexandru Vicol
University of Toronto, Department of Electrical & Computer Engineering
`HealthLog Protocol` — built on Solana + RPC Fast
Draft, July 31, 2026

Abstract

Every consumer neurotechnology and biosignal application re-derives the same thing on first launch: a *per-user baseline*. An EEG brain–computer interface (BCI) fits a personal arousal or intent model; an EMG interface records a maximum-voluntary-contraction (MVC) trial to normalize effort; an HRV app spends days establishing resting norms. These baselines are expensive to collect, physiologically slow to change, and today are trapped inside whichever app produced them. A user who installs a second app starts from zero. There is no portable, verifiable, owner-controlled representation of "my calibration."

We present a design and working implementation that treats a baseline as a first-class, portable asset. The fitted parameters stay encrypted on the user's device; only a SHA-256 *commitment* plus non-identifying shape metadata is anchored on a public ledger (Solana). Any third-party app that later receives the baseline out of band re-hashes it and checks it against the on-chain commitment before trusting it—obtaining integrity, provenance, ownership, and revocation without the raw biometric ever touching the chain. We extend the HealthLog consent protocol (Solana Colosseum, 2026) from *sessions* to *baselines*, using RPC Fast as the mainnet verification rail. We anchor and round-trip-verify real EEG, EMG, and HRV baselines on Solana devnet, show that tampering with a single parameter is detected, measure a median verification read latency of ≈ 106 ms, and enforce the privacy invariant with a runtime gate and automated tests. We close with an honest account of what the current commitment-only design does *not* yet provide—zero-knowledge proofs of baseline quality and cryptographic biometric-template protection—and a roadmap toward them.

1 Introduction

Consumer wearables and BCIs have made biosignals—EEG, EMG, HRV, EDA, PPG—cheap to record. What has *not* become cheap is *calibration*. Because physiology varies across people and electrode placements, almost every serious application begins by estimating a user-specific baseline:

- An EEG focus/relaxation BCI fits a personal model over band-power features (e.g. a Gaussian mixture over θ, α, β and the theta/beta ratio) so that "high arousal" means high *for this user*.
- An EMG gesture or prosthetic interface records an MVC trial and resting RMS per channel, so effort can be expressed as a percentage of that user's own maximum.
- An HRV stress/recovery app accumulates a multi-day resting baseline (SDNN, RMSSD, LF/HF) so a live reading is scored relative to the user rather than to a population.

These baselines share three properties. They are **costly**: minutes of guided recording for EEG/EMG, days for HRV. They are **slow-changing**: useful for weeks to months. And they are **siloed**: each app stores its baseline privately, so the cost is paid again for every new app, device, or research study. A user who has already recorded a perfectly good EEG arousal baseline in app A cannot hand it to app B; app B re-records from scratch, and neither app can prove to a third party (a clinician, a research cohort, a second device) that the baseline it holds is authentic, unmodified, and owned by the person presenting it.

We argue that a baseline should be a *portable, verifiable, owner-controlled object*, and that a public ledger is the right substrate for its *integrity and consent envelope*—though emphatically not for the biometric data itself. Our contributions:

1. A **baseline-commitment design** in which only a SHA-256 hash of the canonicalized parameters plus shape metadata (signal type, model family, feature-schema id, dimensionality, recalibration horizon) is anchored on-chain. The fitted parameters never appear on-chain in any form.
2. An **integration into HealthLog**, extending its Colosseum consent protocol from sessions to baselines, with **RPC Fast** as the mainnet write/verification rail and a free devnet demo rail.
3. A **working reference implementation and evaluation**: real EEG, EMG, and HRV baselines anchored and verified on Solana devnet; single-parameter tamper detection; a runtime privacy gate with automated tests; and read-latency measurements against RPC Fast.
4. An **honest security analysis** that names the residual risks— biometric-template leakage under parameter sharing, linkage across commitments, and the absence of zero-knowledge quality proofs—and a roadmap.

2 Background and Motivation

What a baseline is, concretely. In the HealthLog EEG pipeline, the baseline is a serialized `GmmModelState`: the feature columns, a standardization (mean/scale per feature), Gaussian-mixture weights, means, and covariances, and a mapping from cluster index to an arousal label. This is exactly the object a downstream inference step loads to classify live EEG without re-training. An EMG baseline is a small vector of per-channel MVC and resting-RMS values plus the filter settings used to obtain them; an HRV baseline is a handful of resting time- and frequency-domain statistics with the window over which they were computed. In all three cases the baseline is (i) small (hundreds of bytes to a few kilobytes), (ii) sufficient to make a second app's inference work, and (iii) sensitive, because it is derived from a person's physiology.

Why not just copy the file? A user could in principle export the baseline file and import it elsewhere. Two things are missing. First, *integrity and provenance*: the receiving app has no way to know the file was not altered, or that it belongs to the person presenting it, or when it was recorded and whether it is still fresh. Second, *control*: once copied, the user has no mechanism to revoke a baseline that leaked or to signal that it has been superseded by a recalibration. A public commitment plus a consent envelope supplies exactly these missing pieces without becoming a data-storage system for the biometric.

3 Threat and Trust Model

We assume: (i) the user's device holds the raw baseline and the signing key and is trusted; (ii) the ledger is an untrusted-but-tamper-evident public bulletin board—anything written to it may be read by anyone forever; (iii) recipient apps are semi-trusted—they should receive only the scope the user approves and must be able to *verify* what they receive; and (iv) RPC endpoints (RPC Fast, public devnet) are untrusted for confidentiality but are used only for integrity-checkable reads and for broadcasting already-signed transactions.

The design goals follow directly:

- **Confidentiality:** raw parameters never leave the device except under explicit, scoped user consent, and *never* appear on-chain.
- **Integrity:** a recipient can detect any modification of the baseline it received.
- **Provenance & ownership:** the commitment is bound to the owner's wallet and a timestamp.
- **Revocability:** the owner can invalidate a baseline.
- **Minimality:** the on-chain footprint is a hash and small metadata, nothing more.

4 Design

4.1 The baseline manifest (on-chain envelope)

Anchoring a baseline writes a single memo-program transaction whose payload is a compact JSON envelope. Crucially, the envelope carries a *commitment*, not the data:

```
{ "type": "baseline_commitment", "v": 1,
  "owner": "<wallet>",
  "signal": "eeg",              // eeg|emg|hrv|...
  "model": "gmm-arousal",
  "modelVersion": "1.0.0",
  "featureSchema": "muse-bandpower-4",
  "featureSchemaHash": "9235...e7d13",
  "baselineHash": "509c...da1ca",   // SHA-256 of params
  "dims": { "features": 4, "clusters": 3 },
  "recordedSeconds": 300,
  "validUntil": "2027-07-31T00:00:00Z",
  "ts": "2026-07-31T00:00:00Z" }
```

The `baselineHash` is the load-bearing field. `dims`, `recordedSeconds`, and `validUntil` are deliberately coarse: dimensionality and recording length are context, not identity, and the recalibration horizon lets a verifier reject a stale baseline. `signal` and `model` are short tags so a marketplace of apps can discover which baselines they can consume.

4.2 Canonical commitment

Let P be the fitted parameter object. Define $\mathrm{canon}(P)$ as the recursively key-sorted, whitespace-free JSON serialization of P, and

$$\text{baselineHash} \;=\; \text{SHA-256}\big(\mathrm{canon}(P)\big).$$

Key sorting makes the commitment independent of how a producer happened to assemble the object. Because a baseline anchored by one app must verify in another—possibly written in a different language—canon also pins number formatting: whole-valued numbers render without a decimal point ($2.0 \to 2$, $0.0 \to 0$) and all others via shortest round-trip, so a Dart (Flutter) producer and a Node.js verifier emit byte-identical strings. Without this rule the two diverge—Dart's default encoder renders 0.0 as `"0.0"` where JavaScript renders `"0"`—and the commitments silently mismatch. The same rule is applied to the feature-schema spec to yield `featureSchemaHash`, which lets a consumer confirm it is extracting features the same way the baseline expects—a necessary condition for a baseline to transfer correctly.

4.3 Anchor and verify

The lifecycle mirrors, and reuses, HealthLog's session-consent machinery:

```
Record baseline (once)
  -> canon + SHA-256  -> baselineHash
    -> assert envelope carries no raw params
      -> memo tx (anchor)  ==> tx signature
```

```
------------------------------------------------
Consumer app receives baseline (out of band)
  -> re-hash received params
    -> read commitment at tx (via RPC Fast)
      -> check: type, hash==, owner==, fresh, no-raw
        -> VERIFIED  => load and use; no re-recording
```

Verification is a pure function of the received parameters and the on-chain commitment; it requires no trusted third party. If the received parameters were altered by even one value, the recomputed hash diverges and verification fails.

4.4 Scoped access to the parameters

Anchoring establishes integrity and ownership; it does not move the baseline. Sharing the actual parameters is governed by the same scoped, revocable consent tokens HealthLog already uses for sessions, with a baseline-specific scope ladder: *metadata only* → *feature schema only* → *full baseline parameters*. The most sensitive scope—handing over the fitted model—is opt-in and time-boxed, and the on-chain revocation makes a leaked grant detectable.

4.5 RPC rails: RPC Fast for verification

The value of anchoring is realized at *verification* time, when an app that has never met the user reads the commitment and decides whether to trust a baseline. That read must be fast and reliable, which is why RPC Fast—a low-latency Solana RPC provider—serves as the production verification rail on mainnet. Writing an anchor costs a single memo transaction (≈ 5000 lamports, sub-cent); reading a commitment is a single `getTransaction`. Our free demonstration anchors on devnet (public faucet) so the full mechanism is exercised end-to-end at zero cost; the anchor and verify logic are byte-for-byte identical across networks.

5 Implementation

The implementation has three parts, all in the HealthLog repository.

Flutter app (`lib/`). A `RpcConfig` centralizes the RPC endpoint and is injected at run time via `--dart-define`, so an RPC Fast API key is never committed. New domain models (`WearableBaseline`, `BaselineManifest`) and a `BaselineAnchorService` interface with mock and devnet implementations mirror the existing session/consent services. A *Baseline Passport* screen lets a user load demo EEG/EMG/HRV baselines, anchor them, and run the verification checklist. The commitment reuses the app's existing canonical-JSON hashing so the Dart and reference implementations share one rule.

Reference tool (`tools/baseline_anchor/`). A small, dependency-light Node.js tool (`anchor_baseline.mjs`, `verify_baseline.mjs`) implements the same canonicalization, the same envelope, and a runtime privacy gate that refuses to send any envelope containing a payload-shaped forbidden key (model means, covariances, weights, scaler statistics, raw beat series, MVC vectors, ...) or an overlong numeric array. It reads RPC Fast credentials from a local, git-ignored config and signs with a local keypair. This tool produced every result in §6.

Privacy gate and tests. Both implementations enforce the invariant *"the envelope is a commitment plus shape metadata, nothing more"* at runtime, and an automated test suite asserts that (i) the commitment is deterministic and key-order independent, (ii) no forbidden key or raw value appears in the memo text, (iii) a leaky envelope is rejected, and (iv) a tampered parameter set fails verification.

6 Evaluation

Real anchors. We anchored one baseline of each modality on Solana devnet and verified each round-trip. All six checks (found on-chain; is a baseline commitment; hash matches; owner matches; within horizon; no raw parameters) passed for every baseline.

Signal	`baselineHash` (prefix)	Env. bytes
EEG (gmm-arousal)	`509c8630cca7...`	476
EMG (mvc-normalization)	`2b9f99cd12d7...`	482
HRV (resting-norms)	`929e9e779c58...`	486

Table 1: Baselines anchored on Solana devnet from owner wallet `EKRgJifhv4xs...s4sg`. Envelopes are <500 bytes—well within a single memo transaction. Full signatures are pinned in the repository README and receipts.

Tamper detection binds. Flipping a single GMM weight ($0.42 \rightarrow 0.43$, compensated elsewhere) changed the commitment from `509c8630...` to `bf3eee54...`, and verification against the unchanged on-chain commitment correctly reported FAILED. Integrity is not advisory; it is cryptographically bound to every value in the parameter set.

Cross-language parity. The two independent implementations agree. For all three sample baselines, the Dart (Flutter) hasher and the Node.js reference tool produce byte-identical commitments—enforced as an automated golden test (`test/baseline_parity_test.dart`) that fails if either canonical serializer drifts. This is the property that makes "anchor in app A, verify in app B" real rather than aspirational; before the shared number rule, the same EEG baseline hashed to `a05fdf36...` in Dart versus `509c8630...` in Node.

Verification latency. Over eight `getSlot` probes against the RPC Fast mainnet endpoint we observed 101/106/371 ms (min/median/max), mean 162 ms, dominated by two cold outliers. A full baseline verification is a single `getTransaction` of comparable cost plus local hashing (microseconds), so a consumer app can gate "trust this baseline" on a sub-second, one-round-trip check.

Cost. Anchoring is one memo transaction. On devnet it is free; on mainnet it is the base fee ($\approx$5000 lamports, well under one US cent), paid once per baseline (or per recalibration). Reads are free RPC calls.

7 Security and Privacy Analysis

What is and is not on-chain. On-chain: a hash, an owner wallet, short type/model tags, coarse dimensionality, two timestamps. Off-chain and encrypted: every fitted value, every feature vector, every raw sample. A hash is a one-way commitment; recovering parameters from `baselineHash` is infeasible.

Biometric-template leakage under sharing. The commitment protects the data *on the ledger*, but the sensitive scope of the consent flow does hand the fitted parameters to a recipient. Fitted model parameters can encode identity-linked structure; sharing them is a genuine biometric-disclosure decision, not a metadata one. Our design makes that decision explicit, time-boxed, and revocable, but does not yet make the shared template *cancelable*. Cryptographic template protection (§10) is the right long-term answer; until then the honest framing is: anchoring removes the *integrity/provenance* risk of portability, while parameter sharing remains a consented disclosure.

Linkage and correlation. Multiple commitments from one wallet are publicly linkable, and `signal`/`dims`/`recordedSeconds` leak coarse facts ("this wallet has an EEG and an HRV baseline"). A privacy-seeking user can anchor under rotating or per-baseline wallets at the cost of a unified passport. We treat wallet hygiene as a user-facing choice, surfaced but not mandated.

Revocation semantics. Revocation is a second on-chain event referencing the baseline; a correct verifier treats a baseline whose revocation it can see as untrusted. Because the ledger is append-only, revocation cannot retract data already shared out of band—it can only stop *future* trust. This is the standard limitation of public-ledger consent and we state it plainly.

Freshness and drift. Physiology drifts: an HRV resting baseline ages in weeks, an EEG arousal model more slowly. `validUntil` encodes a recalibration horizon and verification rejects a stale commitment, converting a silent correctness bug (using an outdated baseline) into an explicit, checkable failure.

Key management. The owner wallet is the root of authority. In the demo it is a device-held keypair; a production deployment should use a hardware or mobile-wallet signer. Loss of the key means loss of the ability to revoke or re-anchor, though not disclosure of past data.

8 Limitations and Honest Positioning

We deliberately avoid overclaiming.

- **Commitment, not proof of quality.** The chain attests that a specific parameter set existed and is owned and unmodified. It does *not* attest that the baseline was computed correctly, from enough data, or without bias. "This baseline came from $\geq$5 minutes of clean EEG" is a zero-knowledge claim we do not yet make (§10).
- **Memo, not program state.** The current anchor is an SPL-memo transaction, matching the HealthLog MVP. Structured, queryable state (an Anchor program with per-baseline accounts and native revocation flags) is the intended next step; memos are provenance, not a state machine.
- **Cross-language canonicalization (resolved, with a residual edge).** The Dart and Node implementations now emit byte-identical canonical strings and hashes for realistic baselines, verified by a golden test (§6). The pinned number rule covers normal-range decimals; we do not guarantee agreement for pathological magnitudes (values requiring exponent notation), which do not arise in EEG/EMG/HRV baselines but which a formal spec would still need to fix, e.g. by additionally content-addressing the transmitted byte blob.
- **No cryptographic template protection yet.** As above, the sensitive sharing scope discloses a real template.
- **Devnet demonstration.** Reported anchors are on devnet; the mainnet path is a one-line configuration change and a funded key, and RPC Fast is already the live mainnet read rail we measured against.

9 Related Work

Our design sits at the intersection of self-sovereign identity and biosignal engineering. *Verifiable credentials* and *decentralized identifiers* formalize owner-controlled, verifiable claims; we specialize the pattern to a mutable, drift-bounded biometric baseline and add a recalibration horizon. *Decentralized health-data* efforts typically anchor document hashes or access grants; we anchor a *reusable computational artifact*—a calibration model—and make

its *reuse without re-recording* the goal. From the biosignal side, *transfer learning and cross-session calibration* for BCIs study how to reduce or transport calibration; our contribution is orthogonal and complementary: a portable, verifiable container so a transported baseline can be *trusted* by the receiving system. Finally, *biometric template protection* (cancelable biometrics, fuzzy commitments) is the body of work we point to for hardening the parameter-sharing scope.

10 Future Work

- **Zero-knowledge quality proofs.** Prove "computed from $\geq N$ minutes of artifact-clean signal" or "fit converged" without revealing the data or the parameters, upgrading the commitment from *it exists* to *it is good*.
- **Anchor program with per-baseline accounts.** Native, queryable state and revocation flags; a public index of "baselines this wallet will share, and under what scope."
- **Cancelable-biometric templates.** Share a revocable, non-invertible transform of the baseline so the sensitive scope no longer discloses a reusable template.
- **Cross-app SDK.** A drop-in "verify & load a HealthLog baseline" client so any BCI/wearable app can consume portable baselines, turning the passport into an ecosystem rather than a single app's feature.

11 Conclusion

Baselines are the hidden tax of consumer neurotechnology: costly to collect, slow to change, and trapped in silos. We showed that a baseline can instead be a portable, owner-controlled asset by anchoring only its cryptographic commitment and a thin metadata envelope on a public ledger, keeping the biometric itself encrypted off-chain, and letting any app verify integrity, provenance, and freshness before reuse—no re-recording. The mechanism is implemented, integrated into HealthLog with RPC Fast as the verification rail, and demonstrated end-to-end on real EEG, EMG, and HRV baselines with cryptographic tamper detection and sub-second verification. We were equally explicit about what remains: quality proofs and template protection are future work, and we resist claiming them prematurely. The result is a concrete step toward wearable baselines that people *own* and can *carry*.

References

[1] HealthLog Protocol, "User-owned wearable data with Solana-based consent and provenance," Solana Frontier Hackathon (Colosseum), 2026.

[2] W3C, "Verifiable Credentials Data Model," W3C Recommendation.

[3] W3C, "Decentralized Identifiers (DIDs)," W3C Recommendation.

[4] V. Jayaram et al., "Transfer learning in brain–computer interfaces," *IEEE Computational Intelligence Magazine*, 2016.

[5] N. Ratha, J. Connell, R. Bolle, "Enhancing security and privacy in biometrics-based authentication systems," *IBM Systems Journal*, 2001.

[6] Task Force of ESC/NASPE, "Heart rate variability: standards of measurement, physiological interpretation, and clinical use," *Circulation*, 1996.

[7] A. Yakovenko, "Solana: A new architecture for a high performance blockchain," whitepaper, 2018.